



UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

TÍTULO DE LA MEMORIA/TESIS

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERE CIVIL EN COMPUTACIÓN

NOMBRE NOMBRE APELLIDO APELLIDO

PROFESOR GUÍA:
NOMBRE APELLIDO APELLIDO

PROFESORA CO-GUÍA:
NOMBRE APELLIDO APELLIDO

MIEMBROS DE LA COMISIÓN:
NOMBRE COMPLETO UNO
NOMBRE COMPLETO DOS
NOMBRE COMPLETO TRES

SANTIAGO DE CHILE
2026

RESUMEN DE LA MEMORIA PARA OPTAR AL
TÍTULO DE INGENIERE CIVIL EN COMPUTACIÓN
POR: NOMBRE NOMBRE APELLIDO APELLIDO
FECHA: 2026
PROF. GUIA: NOMBRE APELLIDO APELLIDO

TÍTULO DE LA MEMORIA/TESIS

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Una dedicatoria especial para alguien especial.

Agradecimientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Tabla de Contenido

1. Introducción	1
1.1. Problema abordado	1
1.2. Solución propuesta	1
1.3. Objetivos	2
1.4. Metodología	2
2. Trabajo relacionado	3
3. Diseño del videojuego	4
3.1. Género y mecánicas principales	4
3.2. Diseño de niveles y entornos de prueba	4
3.3. Diseño y arquetipos de enemigos	5
4. Sistema adaptativo basado en el comportamiento del jugador	6
5. Implementación del videojuego	7
5.1. Estadísticas del jugador	9
5.2. Recepción de daño y muerte	10
5.3. Componente de combate	11
5.4. Sistema de input	12
5.5. Ataque melee del jugador	13
5.6. Projectiles	14
5.7. Hechizos y consumibles del jugador	16
5.8. Esquiva	17
5.9. Fijado de objetivo (Target Lock)	18
5.10. Correr	19
5.11. Interactuar	20
5.12. Menú de pausa	20
5.13. Widgets del jugador	20
5.14. Enemigos	22

5.15. Jefe (BP_Slime)	28
5.16. Sistema de métricas y telemetría	31
5.17. Niveles	36
5.18. Widgets generales (MainMenu, WinnerScreen...)	36
6. Prueba de concepto y evaluación	37
7. Conclusión y trabajo futuro	38
Bibliografía	39
A. Anexo	40

Índice de Tablas

Índice de Ilustraciones

Capítulo 1

Introducción

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius.

1.1. Problema abordado

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

1.2. Solución propuesta

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem,

ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

1.3. Objetivos

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

1.4. Metodología

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et.

Capítulo 2

Trabajo relacionado

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim admodum doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Capítulo 3

Diseño del videojuego

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

3.1. Género y mecánicas principales

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum.

3.2. Diseño de niveles y entornos de prueba

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis

philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius.

3.3. Diseño y arquetipos de enemigos

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

asdadaasdas

Capítulo 4

Sistema adaptativo basado en el comportamiento del jugador

Capítulo 5

Implementación del videojuego

Para el proyecto se decidió utilizar Unreal Engine 5.6. El juego se construyó tomando como base la plantilla *Third Person* incluida en Unreal Engine. Esta corresponde a un ejemplo diseñado para demostrar las funcionalidades fundamentales del sistema de personajes en tres dimensiones, incorporando un escenario simple, una cámara orbital y un personaje capaz de desplazarse, correr y saltar. Si bien a primera vista los elementos presentados parecen utilizables directamente para la implementación del proyecto, la plantilla posee diversos aspectos que difieren del objetivo buscado y que requieren modificaciones específicas.

La plantilla *Third Person* proporciona una estructura de proyecto preconfigurada que incluye varios componentes clave. El elemento principal es el `BP_ThirdPersonCharacter`, un Blueprint que representa al personaje jugable y contiene toda la lógica de movimiento, entrada de usuario y comportamiento básico.

A lo largo de esta sección se hará referencia frecuente al concepto de Blueprint. Los Blueprints son el sistema de *scripting* visual de Unreal Engine que permite crear lógica de juego mediante nodos interconectados, sin necesidad de escribir código tradicional. Esto facilita la iteración rápida y permite a diseñadores y artistas contribuir directamente a la programación del juego.

Este Blueprint incluye componentes como la cápsula de colisión, la malla del personaje (*Skeletal Mesh*), el componente de movimiento (*Character Movement Component*) y la cámara con su brazo de resorte (*Spring Arm*). Adicionalmente, la plantilla incluye el *Game Mode*, que define las reglas básicas del juego, y diversos assets como animaciones base, materiales y el escenario de demostración.

Un ejemplo de los ajustes necesarios es la estructura del personaje y sus sistemas de locomoción, los cuales, si bien sirven como referencia inicial, deben ser adaptados o reemplazados según las necesidades mecánicas del juego. Del mismo modo, el comportamiento de la cámara, las animaciones base y la lógica de entrada proporcionadas por defecto funcionan como punto de partida, pero no necesariamente representan la experiencia final deseada.

Por ejemplo, se tuvieron que cambiar las animaciones de movimiento del personaje por algunas que soportaran *Strafe Movement*. El *Strafe Movement* (o simplemente *strafing*) consiste en moverse lateralmente sin perder la orientación hacia un objetivo específico. En lugar de girar todo el cuerpo para caminar hacia la izquierda o la derecha, el personaje desliza su movimiento lateral mientras mantiene la mirada fija hacia adelante o hacia un blanco. Este tipo de movimiento es común en juegos de acción y combate, ya que permite al jugador mantener contacto visual con enemigos mientras se desliza en cualquier dirección.

Se importaron animaciones que soportan el *Strafe Movement*, con las opciones de *Root Motion* y *Enable Root Motion* habilitadas. El *Root Motion* permite que el movimiento del personaje sea controlado directamente por las animaciones en lugar de ser aplicado mediante código, lo que resulta en desplazamientos más naturales y sincronizados con las animaciones reproducidas.

Lo siguiente fue crear un *Blend Space* asociado al esqueleto del jugador, para poder transicionar animaciones dependiendo de la velocidad y dirección del jugador. Un *Blend Space* es un asset de Unreal Engine que permite interpolar suavemente entre múltiples animaciones basándose en valores de entrada. En este caso, se configuró con dos ejes: uno horizontal que representa la dirección lateral del movimiento y uno vertical que representa la velocidad hacia adelante o atrás.

En el *Animation Blueprint* del personaje (*ABP_Manny*), específicamente en el *Animation Graph* dentro de la máquina de estados de *Locomotion*, se reemplazó la animación de *Idle* que venía en el template por una propia, perteneciente al conjunto de animaciones del *Strafe Movement*.

Luego, se reemplazó en el estado de *Walking/Run* el *Blend Space* que venía en el template por el *Blend Space* creado previamente, permitiendo así transiciones fluidas entre las diferentes direcciones de movimiento mientras se mantiene la orientación del personaje. También se desactivó el parámetro «Orient Rotation to Movement» y se activó el parámetro «Use Controller Desired Rotation» para que el jugador mire siempre hacia la dirección que indica la cámara, independientemente de la dirección en la que se mueva.

Cabe recalcar que la mayoría del desarrollo de las implementaciones está realizado en Blueprint. Sin embargo, en el futuro se espera incorporar componentes heredados de C++.

Para desarrollar las capacidades de combate del jugador, se decidió crear un componente `BPC_Combat` que hereda de *Actor Component* y se ancla al Blueprint del jugador, de tal forma que esté centralizada la lógica de combate. De igual forma, para las estadísticas del jugador se creó y ancló un componente `BPC_Stats`.

5.1. Estadísticas del jugador

Como se mencionó anteriormente, las estadísticas del jugador se implementaron en el componente `BPC_Stats` anclado al jugador. Este componente posee múltiples variables, entre las cuales están:

1. `health`: Tipo *Float*. Representa la vida actual del jugador.
2. `Progress Bar Ref`: Tipo *Progress Bar*. Es la referencia a la barra de progreso que muestra la vida.
3. `maxHealth`: Tipo *Float*. Representa el valor máximo de vida del jugador.
4. `stamina`: Tipo *Float*. Representa la stamina actual del jugador.
5. `maxStamina`: Tipo *Float*. Representa la stamina máxima del jugador.
6. `Stam Bar Ref`: Tipo *WB Player Stamina*. Es la referencia al widget que muestra la barra de stamina.
7. `canRecoverStamina`: Tipo *Boolean*. Indica si el jugador puede recuperar stamina.
8. `maxMana`: Tipo *Float*. Representa el valor máximo de maná del jugador.
9. `mana`: Tipo *Float*. Representa el maná actual del jugador.
10. `Mana Bar Ref`: Tipo *Progress Bar*. Es la referencia a la barra que muestra el maná.

La funcionalidad principal de este componente es una función llamada `IncreaseVal`, la cual recibe como parámetros un enum que indica la estadística que se quiere modificar, junto con el valor del cambio (positivo para incrementar, negativo para disminuir). Según el valor del enum, se actualiza la estadística correspondiente sumándole el valor recibido, acotando siempre el resultado entre cero y su valor máximo (`maxHealth`, `maxMana` o `maxStamina`, según corresponda) mediante la función `FClamp`, de modo que ninguna estadística pueda exceder su máximo ni quedar en negativo.

Si la estadística modificada es la vida (`health`), tras actualizarla se realizan dos acciones en paralelo: por un lado, se comprueba si la vida resultante es menor o igual a cero, y de ser así se invoca el evento `Die`, que da inicio a la secuencia de muerte del jugador; por otro, se calcula el porcentaje de vida actual (`health/maxHealth`) y, desde la referencia a la barra (`Progress Bar Ref`), se llama al método `Set Percent`, que actualiza visualmente la barra con dicho porcentaje. En caso de recibir el enum perteneciente a la estadística de maná, el proceso de actualización visual es análogo.

Para la estadística de stamina también es similar, con la diferencia de que el widget de la stamina no se muestra siempre, solo cuando se está gastando o recuperando stamina,

similar a como lo implementan juegos como *The Legend of Zelda: Breath of the Wild*¹.

Así, al recibirse el enum de stamina, se debe revisar si existe un widget creado en pantalla. Si se posee, simplemente se le indica al widget que cambie su porcentaje y al final se revisa si el jugador posee toda la stamina. En caso contrario, significa que el widget no ha sido creado, por lo que se crea y se añade a la pantalla.

Finalmente, se revisa si la stamina es mayor o igual al máximo (100%). En caso de ser verdadero, significa que el jugador ya posee toda la stamina, por lo que se procede a remover el widget de la pantalla.

Al iniciarse el componente este activa dos timers que se ejecutan cada 0.01 segundos y llaman a los eventos `RegainStamina` y `RegainMana`. Estos eventos pasan por una rama que verifica si corresponde regenerar la estadística respectiva; si la condición es verdadera, se usa el nodo `IncreaseVal` para aumentar la stamina o el maná en pequeñas cantidades, creando así un sistema de regeneración continua controlada por condiciones. Es importante mencionar que este enfoque de regeneración mediante timers de alta frecuencia podría optimizarse en el futuro.

5.2. Recepción de daño y muerte

El procesamiento del daño y la curación que recibe el jugador se centraliza en el evento `ReceiveAnyDamage` de `BP_ThirdPersonCharacter`, el cual es invocado automáticamente por el motor cada vez que se llama a la función `ApplyDamage` sobre el jugador, sin importar el origen del daño. Este evento recibe el valor a aplicar y, según su signo, distingue entre dos casos.

Si el valor es negativo, se interpreta como curación: se invoca directamente `IncreaseVal` sobre `BPC_Stats` con el valor invertido, restaurando la vida del jugador según lo descrito en la sección anterior.

Si el valor es positivo, se interpreta como daño recibido, y se ejecuta una secuencia más elaborada. En primer lugar, se intenta identificar si el causante del daño es el jefe (`BP_Slime`); de ser así, se consulta su *Behaviour Tree* para obtener cuál fue su último ataque (`LastAttack`) y se registra en el `PlayerMetricsComponent` mediante `RegisterBossAttackHit`, asociando así cada golpe recibido con el tipo de ataque que lo causó. A continuación, se distingue si el jugador se encontraba esquivando en el momento del impacto (`isDodging`): si es así, se registra como un dodge fallido mediante `RegisterDodgeResult`; en caso contrario, se registra el daño normalmente mediante `RegisterDamageTaken`, y se reproduce el sonido de dolor correspondiente. Si además el jugador se encontraba dentro

¹The Legend of Zelda: Breath of the Wild es un videojuego de acción y aventuras en mundo abierto desarrollado y publicado por Nintendo en 2017. El juego introduce mecánicas innovadoras de supervivencia y exploración, incluyendo un sistema de stamina que se muestra dinámicamente solo cuando se está utilizando o recuperando. Disponible en sitio web.

de una zona específica del combate (`isInTankZone`), se efectúa un registro adicional (`RegisterDamageTakenInTankZone`).

Finalmente, se aplica el daño a la vida del jugador mediante `IncreaseVal` y, si el daño es mayor a cero, se reproduce la animación de reacción a daño (`HitReact_Montage`), que interrumpe temporalmente el movimiento del jugador. Esta interrupción respeta las acciones que ya estén en curso: si el jugador se encuentra en medio de la animación de beber poción, de lanzar un hechizo, o del combo de ataque con espada, el `HitReact_Montage` no detiene el movimiento, evitando que la reacción a daño corte abruptamente otra acción ya iniciada.

Cuando la vida del jugador llega a cero, `IncreaseVal` invoca el evento `Die`, que gestiona la secuencia de derrota. Este evento desactiva el componente de combate, reproduce una animación de derrota (`MO_DefeateDaiko`) y, una vez finalizada, pausa el juego, muestra el widget de «Try Again» y habilita el cursor del mouse. Si la derrota ocurrió en el nivel del jefe, adicionalmente se vuelca a un archivo el registro de resultados del combate mediante `DumpCombatResultToFile`, dato que forma parte del sistema de recolección de métricas del estudio de usuario.

5.3. Componente de combate

Como se mencionó anteriormente, se creó un *Actor Component* llamado `BPC_Combat` que se ancla al jugador y centraliza toda la lógica relacionada con el combate.

Este componente posee múltiples variables, entre las cuales están:

1. `TargetLock`: Tipo *Actor*. Es la referencia al objetivo que se tiene fijado por la cámara.
2. `Player BP`: Tipo *Object Reference* a *BP Third Person Character*. Referencia al jugador.
3. `TargetLockWidget`: Tipo *Object Reference* a *BP Target Lock Widget*. Es la referencia al widget del símbolo que se crea cuando se fija un objetivo.
4. `isAttacking`: Tipo *Boolean*. Es una bandera que se levanta cuando el jugador está atacando.
5. `isCombo`: Tipo *Boolean*. Bandera que se levanta cuando el jugador tiene la intención de continuar el ataque.
6. `isDodging`: Tipo *Boolean*. Bandera que se levanta cuando el jugador está realizando una esquivas.
7. `StamWidget`: *Object Reference* a *WB Player Stamina*. Es la referencia al widget de la stamina que se muestra cuando se gasta o recupera stamina.
8. `StaminaRollUse`: Tipo *Float*. Define la cantidad de stamina que consume la acción de rodar.
9. `isRunning`: Tipo *Boolean*. Bandera que se levanta cuando el jugador está corriendo.
10. `StaminaRunUse`: Tipo *Float*. Define la cantidad de stamina que gasta el jugador al correr.
11. `runSpeed`: Tipo *Float*. Define la velocidad del jugador al correr.

12. `CurrentSpellIndex`: Tipo *Float*. Índice que indica el hechizo que tiene equipado el jugador actualmente.
13. `SpellImageObject`: Tipo *Image*. Referencia a la imagen que indica el hechizo equipado actualmente en la interfaz.
14. `magicManaUse`: Tipo *Float*. Define la cantidad de maná que consume el proyectil mágico del jugador.
15. `healingManaUse`: Tipo *Float*. Define la cantidad de maná que consume el hechizo de curación.
16. `SwordDamage`: Tipo *Float*. Define el daño base que inflige la espada del jugador.
17. `currentLever`: Tipo *Object Reference* a *BP Lever*. Referencia a la palanca con la que el jugador está interactuando actualmente.
18. `isEnemyDashing`: Tipo *Boolean*. Bandera que indica si el enemigo está realizando una embestida o desplazamiento rápido.
19. `isInCorrectDodgeWindow`: Tipo *Boolean*. Bandera que indica si el jugador se encuentra dentro de la ventana temporal adecuada para ejecutar una esquivada efectiva.
20. `isRollingSideWays`: Tipo *Boolean*. Bandera que indica si la esquivada actual se está realizando lateralmente.
21. `MovInput`: Tipo *Vector2D*. Almacena la dirección de movimiento introducida por el jugador.
22. `FireBallImage`: Tipo *Image*. Referencia a la imagen utilizada en la interfaz para representar el hechizo de bola de fuego.
23. `PotionImage`: Tipo *Image*. Referencia a la imagen utilizada en la interfaz para representar las pociones del jugador.
24. `PotiCount`: Tipo *Integer*. Indica la cantidad actual de pociones disponibles.
25. `Potion Count Text Var`: Tipo *Text*. Referencia al elemento de texto de la interfaz que muestra la cantidad de pociones disponibles.
26. `healingPotion`: Tipo *Static Mesh*. Referencia al modelo 3D de la poción de curación utilizada por el jugador.
27. `tmpTarget`: Tipo *Vector*. Almacena una ubicación temporal utilizada por los enemigos como objetivo durante la ejecución de sus comportamientos.

5.4. Sistema de input

La entrada del jugador se gestiona mediante el sistema *Enhanced Input* de Unreal Engine. Cada acción del jugador (atacar, esquivar, correr, etc.) se representa como un *Input Action* independiente, desacoplado de la tecla o botón físico que lo activa. Esta asociación entre la entrada física y el *Input Action* correspondiente se define en un *Input Mapping Context*, lo que facilita el remapeo de controles sin modificar la lógica interna de cada Blueprint. Actualmente el juego solo cuenta con soporte para teclado y mouse.

Los *Input Actions* utilizados por el componente de combate (*BPC_Combat*) y sus controles asociados son los siguientes:

Input Action	Control
IA_Attack	Click izquierdo
IA_Dash-Roll	Barra espaciadora
IA_Run	Shift
IA_Magic	Q
IA_ScrollMagic	Rueda del mouse
IA_Lock	Click rueda del mouse
IA_MyInteract	E
IA_EscapeMenu	Escape

En las secciones siguientes, cada acción del jugador se describe haciendo referencia explícita al **Input Action** correspondiente (por ejemplo, **IA_Attack** o **IA_Dash-Roll**), ya activado mediante el *Input Mapping Context* descrito anteriormente.

5.5. Ataque melee del jugador

El ataque del jugador se implementó en el componente de combate mencionado previamente. Al recibirse el **Input Action** **IA_Attack**, antes de ejecutar la acción se comprueba que el jugador no esté ya atacando ni esquivando (variables **isAttacking** e **isDodging**) y, además, que no se esté reproduciendo el *Animation Montage* de reacción a daño (**HitReact_Montage**), de modo que el jugador no pueda atacar mientras recibe un golpe. Si la condición se cumple, se setea **isAttacking** como **true**, se registra el ataque cuerpo a cuerpo en el *Game Instance* (para fines de telemetría) y se ejecuta el *Animation Montage* asociado al ataque, que contiene una secuencia de tres golpes consecutivos.

El encadenamiento de golpes (combo) se controla mediante la variable **isCombo**, que representa la intención del jugador de continuar la secuencia. Esta variable se activa de forma diferida: si el jugador vuelve a recibir el **Input Action** **IA_Attack** mientras un ataque ya está en curso (es decir, cuando la condición inicial es **false** porque **isAttacking** es **true**), en lugar de iniciar un ataque nuevo se setea **isCombo** como **true**. De esta forma el sistema «almacena» la intención de combo hasta el siguiente punto de decisión.

Dicho punto de decisión se evalúa mediante un *Animation Notify* ubicado en el montage, al final del primer y segundo golpe. Cada vez que el notify se activa, se comprueba si el jugador sigue atacando y si tiene la intención de continuar (variables **isAttacking** e **isCombo**). Si ambas son verdaderas, se permite que la secuencia continúe y se resetea **isCombo** a **false** (a la espera de un nuevo input). En caso contrario, se detiene el montage prematuramente mediante **Montage_Stop** y se limpian las variables **isAttacking** e **isCombo**. Cuando el montage se completa o es interrumpido, se restablecen también **isAttacking** e **isCombo** a **false** y se restaura el estado de movimiento del personaje; en el caso de interrupción, se distingue además si la causa fue la reproducción del **HitReact_Montage**.

Con respecto a la detección de impacto del ataque, la implementación se basa en un sistema de *trace* por temporizador. El montage tiene asociado un Blueprint que hereda de *Animation Notify State*, el cual delimita la ventana en la que el arma debe poder golpear mediante un momento de inicio y un momento de fin dentro del montage. Al recibirse la señal de inicio, se invoca el evento **Begin Damage Trace**, que inicia un *timer* en bucle que, cada 0.1 segundos, ejecuta un *Sphere Trace* entre dos componentes de escena anclados al arma (**Start Sword Trace Pos** y **End Sword Trace Pos**), que marcan la base y la punta de la hoja. Si el *trace* detecta una colisión válida, se llama a la función **ApplyDamage** sobre el actor impactado, utilizando el valor de la variable **SwordDamage** y al jugador como causante del daño. Al recibirse la señal de fin, se invoca el evento **End Damage Trace**, que invalida el *timer*, desactivando la detección hasta el siguiente golpe.

5.6. Projectiles

Tanto el jugador como el jefe pueden lanzar proyectiles. Es por esto que se decidió crear un Blueprint Class que represente la base de un proyectil (**BP_BaseProjectile**), para que después tanto el proyectil del jugador como el del jefe hereden desde esta clase base y ajusten variables específicas de cada uno. El comportamiento por defecto del proyectil es avanzar con velocidad constante en línea recta hasta chocar con algo, momento en el cual reproduce el efecto visual (VFX) y el sonido de impacto correspondientes, aplica daño al objetivo impactado y se destruye.

El proyectil base posee múltiples componentes, entre los cuales están:

1. **BoxCollision**: Componente de colisión (*Box Collision*). Define el volumen de colisión principal del proyectil.
2. **Arrow**: Componente *Arrow*. Sirve como referencia visual y direccional para indicar hacia dónde se moverá u orientará el proyectil.
3. **StaticMesh**: Componente *Static Mesh*. Representa la malla estática visible del proyectil, en caso de necesitarse.
4. **ProjectileMovement**: Componente *Projectile Movement*. Controla el movimiento del proyectil, incluyendo velocidad, gravedad y comportamiento de persecución (*homing*).

Las variables de las que se compone son:

1. **Speed**: Tipo *Float*. Define la velocidad a la que se mueve el proyectil.
2. **Gravity**: Tipo *Float*. Controla cuánta influencia tiene la gravedad sobre el proyectil.
3. **Homing**: Tipo *Boolean*. Indica si el proyectil tiene la capacidad de perseguir automáticamente a un objetivo.
4. **Base Damage**: Tipo *Float*. Define el daño base que causa el proyectil al impactar.
5. **Impact Effect**: Tipo *Particle System*. Es la referencia al sistema de partículas que se reproduce cuando el proyectil impacta.
6. **Sound Impact**: Tipo *Sound Base*. Es la referencia al sonido que se reproduce cuando el proyectil impacta.

El comportamiento del proyectil base se define mediante los siguientes eventos. Al inicializarse (**BeginPlay**), el proyectil configura su componente de colisión para ignorar al actor que lo originó (**GetOwner**), evitando así que colisione consigo mismo o con quien lo disparó. Además, si la variable **Homing** está activada, se habilita el modo de persecución del componente *Projectile Movement* (**bIsHomingProjectile**) y se establece el componente objetivo de la persecución.

La detección de impacto se gestiona mediante un evento ligado al solapamiento del componente de colisión. Cuando el proyectil se solapa con otro actor, se comprueba que dicho actor no sea su propio dueño (**GetOwner**); de no serlo, se invoca la función **SpawnImpactEffect**, que reproduce el sistema de partículas **Impact Effect** en el punto de colisión junto con el sonido **Sound Impact**. A continuación se aplica el daño definido en **Base Damage** sobre el actor impactado —indicando al dueño del proyectil como causante del daño— y finalmente el proyectil se destruye.

5.6.1. Proyectil del jugador

Uno de los hechizos que puede utilizar el jugador es el de disparar un proyectil. El Blueprint que representa al proyectil del jugador (**BP_PlayerFireBall**) hereda de **BP_BaseProjectile** y configura las siguientes variables: **Speed** con un valor alto (para que sea más rápido que el proyectil del jefe), **Gravity** en 0 (sin influencia gravitacional), **Homing** como **false** (no persigue objetivos), y el **Impact Effect** con un VFX de explosión de fuego. En cuanto a los componentes visuales, el **StaticMesh** permanece vacío y en su lugar se utiliza un componente *Particle System* que representa el efecto visual del proyectil en movimiento.

5.6.2. Proyectil del jefe

El jefe puede lanzar proyectiles mediante ciertos ataques. El Blueprint que los representa (**BP_BossSlimeBall**) hereda de **BP_BaseProjectile** y configura sus variables de la siguiente manera: **Speed** con un valor moderado (menor que el del jugador), **Gravity** en 0, el **Impact Effect** con un VFX de explosión de slime, y **Homing** que varía según el tipo de ataque. El jefe dispone de dos variantes de ataque con proyectiles: uno donde el proyectil avanza en línea recta a velocidad constante (**Homing** en **false**), y otro donde persigue activamente al jugador hasta alcanzarlo o explotar tras un tiempo determinado (**Homing** en **true**). Al igual que el proyectil del jugador, el **StaticMesh** permanece vacío y se emplea un componente *Particle System* para el efecto visual del proyectil en movimiento.

Esta subclase sobrescribe (*override*) el evento **BeginPlay** para extender el comportamiento heredado: al instanciarse, primero ejecuta el **BeginPlay** de la clase base (configurando la colisión y el *homing* según corresponda) y, a continuación, inicia un temporizador de 5 segundos. Transcurrido ese tiempo, si el proyectil aún no ha impactado contra nada, spawnear un sistema de partículas **Niagara (NS_Projectile_03_Hit)** en su ubicación actual y se autodestruye, garantizando así que los proyectiles que no alcancen al jugador no permanezcan indefinidamente en la escena.

5.7. Hechizos y consumibles del jugador

El jugador dispone de dos opciones que puede alternar y utilizar mediante una misma acción: un hechizo de proyectil de fuego y una poción de vida. Esta mecánica se implementó con el propósito de otorgar al jugador versatilidad en combate —pudiendo elegir entre una opción ofensiva y una defensiva— y de proporcionar al jefe la oportunidad de demostrar capacidad de aprendizaje y adaptación a las diferentes estrategias del jugador.

El jugador puede intercambiar entre el hechizo de fuego y la poción utilizando la rueda del mouse, y la opción actualmente equipada se muestra representada con un símbolo en la esquina inferior derecha de la pantalla. La selección se representa mediante la variable `CurrentSpellIndex`, que cambia al mover la rueda del mouse (`IA_ScrollMagic`); además de actualizar el valor, se modifica el ícono y la opacidad de la imagen del HUD para reflejar la opción activa.

Cuando el jugador ejecuta la acción de lanzar (`IA_Magic`), el sistema evalúa la variable `CurrentSpellIndex` para determinar qué opción está equipada. Si corresponde al hechizo de fuego, se comprueba primero que se disponga del maná necesario (`magicManaUse` frente a la variable `mana`) antes de invocar el evento del proyectil; si corresponde a la poción, se invoca directamente el evento de curación, cuya disponibilidad depende de la cantidad de pociones que posea el jugador.

5.7.1. Hechizo de proyectil de fuego

El evento del hechizo de proyectil (`ProjMagicEvent`) comienza comprobando que el jugador no esté atacando ni esquivando, y que no se esté reproduciendo la animación de reacción a daño. Si la condición se cumple, registra el ataque a distancia en el *Game Instance* con fines de telemetría, marca al jugador como atacando y reproduce la animación de lanzamiento, que corresponde a un *Animation Montage*. Durante la reproducción se desactiva temporalmente el movimiento del jugador, ya que mientras lanza el proyectil no debe poder desplazarse; el movimiento se restablece cuando la animación finaliza o es interrumpida.

El proyectil en sí se genera mediante un *Animation Notify* anclado al montage: en el instante en que la animación lo indica, se instancia el proyectil del jugador (`BP_PlayerFireBall`) con el jugador asignado como su dueño (*Owner*), de modo que el proyectil no colisione con quien lo dispara. En ese mismo momento se reproduce el sonido de lanzamiento y se realiza el consumo de maná, llamando a la función `IncreaseVal` del componente `BPC_Stats`, pasando como parámetro el valor de la variable `magicManaUse` con signo negativo para representar la disminución.

5.7.2. Poción de vida

La poción de vida constituye la opción defensiva del jugador. No restaura la vida de forma instantánea, sino que requiere que el jugador beba una poción de una cantidad limitada que posee, representada por la variable `PotiCount` y mostrada en el HUD. El evento correspondiente (`HealingMagicEvent`) comienza comprobando que el jugador no esté

atacando ni esquivando, que no se esté reproduciendo la animación de reacción a daño y, fundamentalmente, que disponga de al menos una poción (`PotiCount` mayor que cero).

Si la condición se cumple, se registra el momento de curación en el *Game Instance* (junto con la vida actual y máxima del jugador), se decrementa el contador de pociones y se actualiza el HUD, y se desactiva el movimiento mientras se reproduce la animación de beber, que corresponde a un *Animation Montage*. Para reforzar visualmente la acción, durante la animación se genera y se ancla la malla de la poción a la mano del jugador, la cual se destruye una vez finalizada. La restauración de vida se realiza mediante un *Animation Notify* anclado al montage: en el momento en que la animación muestra al jugador bebiendo, se llama a la función `IncreaseVal` para restaurar la vida del jugador. Al completarse o interrumpirse la animación, se restablece el movimiento del jugador.

5.8. Esquiva

La esquiva del jugador está implementada en el componente `BPC_Combat`. Se creó una función auxiliar (`GetRollMontage`) para determinar qué animación de roll ejecutar, ya que se puede realizar un roll hacia adelante, hacia atrás, o hacia los lados cuando se tiene un objetivo fijado mediante el sistema de *Target Lock*.

La función `GetRollMontage` determina cuál de las cuatro animaciones de roll ejecutar a partir de la dirección de movimiento del jugador. Para ello construye el vector de intención de movimiento en espacio mundo, combinando la entrada de movimiento (`MovInput`, actualizada continuamente mediante el `Input Action IA_Move`) con la rotación de la cámara (la rotación del controlador), y lo normaliza. Luego proyecta ese vector sobre el *Forward Vector* y el *Right Vector* del personaje mediante productos punto, y compara las magnitudes de ambas proyecciones: si la componente hacia adelante/atrás es mayor o igual que la lateral, el roll se considera frontal; en caso contrario, lateral.

Una vez determinado el eje dominante, el signo del producto punto correspondiente define el sentido exacto y, con ello, el *Animation Montage* que se retorna: hacia adelante (`Dodge_F_MontageDAIKO`) o hacia atrás (`Dodge_B_MontageDAIKO`) en el caso frontal, y hacia la derecha (`Dodge_R_MontageDAIKO`) o hacia la izquierda (`Dodge_L_MontageDAIKO`) en el caso lateral. Cuando el roll es lateral, además se marca la variable `isRollingSideWays` como `true`.

Adicionalmente, esta función realiza un registro de telemetría específico para las esquivas realizadas frente a una embestida del jefe: si en el momento del roll el jefe se encuentra embistiendo (`isEnemyDashing`) y el jugador está dentro de una zona determinada (`isInTankZone`), se llama a la función `RegisterDashDodge` del *Game Instance*, pasándole el parámetro `bWasLateral` para indicar si la esquiva fue lateral (`true`, en los rolls hacia los lados) o frontal (`false`, en los rolls hacia adelante o atrás). Esto permite recopilar métricas diferenciadas sobre cómo responde el jugador ante el ataque de embestida según la dirección de su evasión.

En cuanto a la lógica del roll: cuando se recibe el `Input Action IA_Dash-Roll`, primero se comprueba que el jugador no esté atacando (variable `isAttacking`), que disponga de la stamina suficiente para realizar la esquiva (comparando la `stamina` actual contra `StaminaRollUse`) y que no se esté reproduciendo la animación de reacción a daño (`HitReact_Montage`). Si la condición se cumple, se registra la esquiva en el *Game Instance* mediante la función `RegisterTotalDodge` y se procede a ejecutar la acción: se llama a la función `IncreaseVal` para restar la stamina consumida, se marca `isDodging` como `true`, se reproduce el sonido de esquiva y se ejecuta la animación de roll correspondiente (obtenida mediante `GetRollMontage`). Inmediatamente se setea la variable `canRecoverStamina` como `false` para evitar que se regenere stamina durante la esquiva.

La limpieza de los estados se realiza al término de la animación de roll. Tanto si el montage se completa exitosamente como si es interrumpido, se restablece `canRecoverStamina` a `true` (permitiendo nuevamente la regeneración de stamina) y se limpian las variables `isDodging` e `isRollingSideWays`, devolviendo al jugador a un estado neutro desde el cual puede volver a actuar.

Adicionalmente, el sistema distingue si la esquiva se realizó dentro de una ventana de tiempo considerada como una evasión precisa o «perfecta», mediante la variable `isInCorrectDodgeWindow`. Cuando este es el caso, se efectúa un registro extra en el *Game Instance* mediante la función `RegisterDodgeFromDelayAttack`, lo que permite recopilar métricas sobre las esquivas realizadas con buen *timing* frente a los ataques enemigos.

Cabe mencionar que todo esto se ejecuta dentro de un nodo *Do Once*, el cual, como bien indica su nombre, solo se ejecuta una vez hasta que se resetee. El reseteo se realiza después de 0.45 segundos mediante un nodo *Delay*. Esto permite que el jugador pueda hacer un roll cada 0.45 segundos, pudiendo así encadenar rolls consecutivos para evadir múltiples ataques.

5.9. Fijado de objetivo (Target Lock)

El sistema de fijado de objetivo (*Target Lock*) permite al jugador centrar la cámara sobre un enemigo y mantenerla orientada hacia él durante el combate, facilitando el seguimiento del objetivo mientras se ataca o esquiva. Está implementado en el componente `BPC_Combat` y se reparte entre dos partes: la activación/desactivación del fijado, gestionada por el `Input Action IA_Lock`, y el mantenimiento de la orientación de la cámara, ejecutado en cada fotograma dentro del evento `Tick`.

Al recibirse el `Input Action IA_Lock`, el sistema actúa según si ya existe un objetivo fijado, almacenado en la variable `Target Lock`. Si ya hay un objetivo, se interpreta como una orden de soltarlo: se limpia la referencia, se restauran los parámetros de rotación del personaje a su comportamiento normal (orientación según el movimiento) y se destruye el widget visual del fijado. Si no hay objetivo, se procede a buscar uno: se realiza un *Sphere Trace* hacia adelante desde el personaje (con un radio y un alcance determinados);

si dicho trazado detecta un objetivo válido, se almacena en `Target Lock`, se ajustan los parámetros de rotación para que el personaje se oriente hacia el objetivo, y se genera un widget (`BP_TargetLockWidget`) que se ancla al enemigo como indicador visual del fijado.

Una vez fijado el objetivo, su seguimiento se realiza en el evento `Tick`. Mientras exista un objetivo, cada fotograma se calcula la rotación necesaria para que la cámara apunte hacia él y se aplica a la rotación del controlador del jugador. Para obtener el punto exacto al que mirar, el sistema comprueba si el objetivo implementa la interfaz `BPI_Lockable`: si la implementa, solicita a través de ella un punto de fijado personalizado, lo que permite que ciertos enemigos —como aquellos cuya geometría se deforma o cuyo centro visual no coincide con su origen— definan explícitamente dónde debe apuntar la cámara; si no la implementa, se utiliza directamente la ubicación del objetivo. Además, mientras el fijado está activo, se reposiciona en pantalla el widget de la barra de stamina.

Actualmente, la única clase que implementa la interfaz `BPI_Lockable` es el jefe (`BP_Slime`). Esto se debe a que su animación de ataque de charco desplaza visualmente al personaje hacia abajo —simulando que se hunde o agacha en el charco— sin que dicho desplazamiento se traduzca en un movimiento real del actor en el mundo. Si la cámara utilizara directamente la ubicación del actor (su origen), el punto de fijado quedaría desalineado respecto a la posición visual del jefe durante esta animación. Al implementar `BPI_Lockable`, el jefe puede reportar un punto de fijado ajustado a su posición visual real en cada momento, manteniendo la cámara correctamente orientada incluso cuando la animación no coincide con la posición lógica del actor.

5.10. Correr

La carrera permite al jugador desplazarse a mayor velocidad a cambio de consumir stamina, y está implementada en el componente `BPC_Combat` mediante el `Input Action IA_Run`. A diferencia de otras acciones, esta responde a las distintas fases del input —el momento en que se presiona, mientras se mantiene presionado, y cuando se suelta—, lo que permite controlar tanto la activación como el gasto continuo de stamina y el regreso al estado normal.

Como condición común a todas las fases, la carrera solo opera si no se está reproduciendo la animación de reacción a daño (`HitReact_Montage`), de modo que recibir un golpe interrumpe la posibilidad de correr.

Al presionar el input, si el jugador dispone de stamina suficiente, se aumenta su velocidad de movimiento (`MaxWalkSpeed`) al valor definido por la variable `runSpeed` y se bloquea la regeneración de stamina (`canRecoverStamina` en `false`), ya que no debe recuperarse stamina mientras se corre. Mientras el input se mantiene presionado, y siempre que aún quede stamina, se consume de forma continua llamando a la función `IncreaseVal` del componente `BPC_Stats` con el valor de `StaminaRunUse` en negativo, lo que reduce progresivamente la stamina disponible durante la carrera. Finalmente, al soltar el input, se restaura la velo-

cidad de movimiento a su valor normal y se vuelve a habilitar la regeneración de stamina, devolviendo al jugador a su estado de desplazamiento habitual.

5.11. Interactuar

La interacción del jugador con elementos del entorno está implementada en el componente `BPC_Combat` mediante el `Input Action IA_MyInteract`. Actualmente esta acción está orientada a la interacción con palancas: al recibirse el input, primero se comprueba que exista una palanca con la que el jugador pueda interactuar en ese momento, mediante la referencia `currentLever`. Si existe dicha referencia, se procede a verificar si ya fue utilizada (`bAlreadyUsed`); de no haberlo sido, se invoca la función `PlayLever` sobre el Blueprint de la palanca (`BP_Lever`), que se encarga de reproducir su animación y de ejecutar el efecto asociado a su activación.

5.12. Menú de pausa

El acceso al menú de pausa está implementado en el componente `BPC_Combat` mediante el `Input Action IA_EscapeMenu`. Al recibirse el input, se comprueba si el juego ya se encuentra pausado; si no lo está, se procede a abrirlo: se instancia el widget del menú de pausa (`WB_PauseMenu`) y se añade al *viewport*, se cambia el modo de entrada a uno orientado exclusivamente a la interfaz (de forma que el movimiento del mouse deje de controlar la cámara), se habilita la visibilidad del cursor, y finalmente se pausa el juego.

5.13. Widgets del jugador

La interfaz de usuario asociada al jugador se compone de varios *Widget Blueprints*, cada uno encargado de mostrar un aspecto específico del estado del jugador o de gestionar una pantalla concreta. La mayoría de estos widgets se actualizan desde la lógica de juego ya descrita —principalmente desde el componente `BPC_Stats`—, por lo que su Blueprint interno es mínimo y se limitan a su composición visual.

El widget principal es el HUD del jugador (`WB_PlayerHUD`), que actúa como contenedor de los elementos que se muestran de forma persistente durante el juego. Está compuesto por un *Canvas Panel* que agrupa la barra de vida (`WB_PlayerHealth`), la barra de maná (`WB_PlayerMana`) y el widget de equipamiento (`WB_Equips`). Este widget no posee lógica en su grafo de eventos, ya que su única función es disponer espacialmente en pantalla a los sub-widgets que contiene.

Los widgets de barra de vida (`WB_PlayerHealth`) y de maná (`WB_PlayerMana`) son estructuralmente simples: cada uno se compone de un *Canvas Panel* que contiene un borde decorativo y una barra de progreso (*Progress Bar*). No poseen lógica en su grafo de eventos, ya que su valor se actualiza directamente desde la función `IncreaseVal` de `BPC_Stats`, la cual modifica el porcentaje de la barra cada vez que cambia la estadística correspondiente, tal como se describió en la sección de estadísticas del jugador.

El widget de equipamiento (**WB_Equips**) muestra los consumibles y hechizos disponibles del jugador. Se compone de una caja horizontal con la imagen del hechizo de fuego (**Fireball_IMG**) y, superpuestos, la imagen de la poción (**Potion_IMG**) junto con un texto que indica la cantidad de pociones restantes (**PotionCountText**). Tampoco posee lógica en su grafo, pues tanto el ícono activo como el contador se actualizan desde **BPC_Combat**.

Finalmente, el widget de fijado de objetivo (**WB_TargetLock**) consiste únicamente en una imagen (un ícono) que se ancla sobre el enemigo fijado, según lo descrito en la sección de *Target Lock*.

5.13.1. Barra de stamina (WB_PlayerStamina**)**

A diferencia de las barras de vida y maná, la barra de stamina sí posee lógica propia en su Blueprint, ya que se representa mediante un material en lugar de una *Progress Bar* estándar. El widget se compone de una única imagen (**Image_176**) a la que se le aplica un material dinámico, lo que permite representar la barra de stamina con una forma circular.

La actualización del nivel de stamina se realiza mediante la función **SetPercent**. Esta función primero comprueba si la instancia dinámica del material (**RoundStaminaBarInst**) ya fue creada: si es válida, simplemente actualiza el parámetro escalar **Percent** del material con el nuevo valor. Si aún no existe, crea una instancia dinámica a partir del material base (**M_StaminaBar_Inst**), la asigna como *brush* de la imagen, y luego actualiza su parámetro **Percent**. De esta forma, la instancia del material se crea una sola vez (la primera vez que se actualiza la barra) y se reutiliza en las llamadas posteriores.

5.13.2. Widget del menú de pausa (WB_PauseMenu**)**

El widget del menú de pausa presenta dos botones dispuestos verticalmente: reanudar (**ResumeBtn**) y volver al menú principal (**MainMenuButton**). Su lógica responde a los eventos de pulsación de cada botón. Al presionar el botón de reanudar, se reanuda el juego (quitando la pausa), se restablece el modo de entrada a uno orientado exclusivamente al juego, se oculta el cursor del mouse, y se elimina el propio widget de la pantalla, devolviendo al jugador al combate. Al presionar el botón de menú principal, se reanuda el juego y se carga el nivel del menú principal (**Lvl_MainMenu**).

5.13.3. Pantalla de derrota (WB_TryAgain**)**

Esta pantalla se muestra cuando el jugador es derrotado e incluye un mensaje de «Game Over» junto con dos botones: reintentar (**Retry Button**) y volver al menú principal (**Main Menu Button**). Al presionar reintentar, se reanuda el juego, se incrementa el contador de intentos (**AttemptNumber**) almacenado en el **SlimeGameInstance** —dato relevante para el seguimiento del estudio— y se recarga el nivel actual, permitiendo al jugador volver a enfrentar el combate. Al presionar el botón de menú principal, se reanuda el juego y se carga el nivel del menú principal.

5.14. Enemigos

5.14.1. Esqueleto normal

El enemigo más simple del juego es el esqueleto normal (`BP_Skeleton`), un enemigo de combate cuerpo a cuerpo cuya estructura de componentes y comportamiento sientan la base sobre la cual se construyen los demás enemigos. Se compone de una cápsula de colisión como componente raíz, sobre la cual se ancla un componente de barra de vida (*Widget Component*); una malla esquelética (`Mesh`) que incluye dos componentes de escena (`StartOfTrace` y `EndOfTrace`) ubicados en la espada, utilizados para la detección de impacto del ataque, y un componente *Arrow* como referencia direccional; un componente de movimiento de personaje; y un componente `PawnSensing`, encargado de la detección del jugador.

5.14.1.1. Comportamiento general

Durante el juego (`ReceiveTick`), la barra de vida rota constantemente para orientarse hacia la cámara del jugador, de modo que siempre se muestre de frente independientemente de la posición del enemigo.

Al iniciarse (`ReceiveBeginPlay`), el esqueleto inicializa su barra de vida al 100% y almacena su posición de aparición en la *Blackboard key spawnPoint*, la cual utiliza posteriormente para regresar a su punto de origen.

La detección del jugador se gestiona mediante el evento `On See Pawn` del componente `PawnSensing`. Al detectar al jugador, el esqueleto invoca una función auxiliar (`hasseen`), la cual castea el Pawn detectado a `BP_ThirdPersonCharacter` —validando que corresponda efectivamente al jugador— y, de ser así, marca la *Blackboard key hasSeenPlayer?* como `true`. Esta misma función auxiliar es invocada también desde el evento de recepción de daño, descrito a continuación.

Adicionalmente, al detectar al jugador se inicia un temporizador en bucle de 0.5 segundos que dispara el evento `REGISTER_DISTANCE`, el cual obtiene la referencia al `SlimeGameInstance` e invoca `RegisterDistance`, pasando como parámetro la distancia actual entre el esqueleto y el jugador. Este mecanismo —compartido con el esqueleto caballero, descrito más adelante— es el que alimenta las variables de distancia de la sección de métricas y telemetría durante la fase previa al combate contra el jefe.

5.14.1.2. Recepción de daño y muerte

El procesamiento de daño se centraliza en el evento `ReceiveAnyDamage`. Al recibir daño, en primer lugar se invoca `hasseen` sobre el causante del daño, de modo que un golpe del jugador también provoca que el esqueleto lo detecte (incluso si no lo había visto previamente). A continuación, se resta el daño recibido a la variable `health`, y se actualiza el porcentaje de la barra de vida correspondiente (`health / maxHealth`).

Si la vida resultante es menor o igual a cero, el esqueleto reproduce un sonido de muerte (*SkeleDeath*) y se destruye. Adicionalmente, se obtiene la referencia al jugador (*BP_ThirdPersonCharacter*) y se invoca sobre él la función *RemoveWidget*, encargada de eliminar el indicador visual asociado a este enemigo (por ejemplo, el del sistema de *Target Lock*, en caso de que estuviera fijado sobre él al momento de morir).

Internamente, *RemoveWidget* limpia la variable *Target Lock* del jugador y obtiene la referencia al widget asociado (*TargetLockWidget*), al cual destruye mediante *K2_DestroyActor*. A continuación, restablece las variables de movimiento del jugador (*bOrientRotationToMovement* a *true* y *bUseControllerDesiredRotation*), revirtiendo el cambio de orientación aplicado mientras el enemigo estaba fijado como objetivo, y finalmente destruye el componente del widget (*K2_DestroyComponent*).

5.14.1.3. Ataque

Al igual que en el ataque del jugador, el inicio y fin de la ventana de detección de impacto están delimitados por un *Anim Notify State* anclado al montage de ataque, el cual invoca los eventos *BegomSwordTrace* y *EndSwordTrace* al comenzar y terminar dicha ventana respectivamente.

El ataque del esqueleto utiliza el mismo patrón de detección por temporizador que el ataque del jugador, aunque con una diferencia en su mecanismo de corte. Al iniciar el ataque (*BegomSwordTrace*)², se reproduce el sonido de ataque (*SFX_SwordSkele*) y se inicia un temporizador en bucle cada 0.01 segundos que ejecuta el evento *Damage Trace*. Este evento realiza un *Sphere Trace* de radio 30 entre los componentes *StartOfTrace* y *EndOfTrace*; si detecta una colisión válida, aplica el daño definido en la variable *skeleton_damage* sobre el actor impactado y, a diferencia del ataque del jugador, detiene inmediatamente el temporizador mediante *K2_ClearAndInvalidateTimerHandle*, deteniendo la detección tras el primer impacto exitoso. Al finalizar la animación de ataque (*EndSwordTrace*), se invalida el temporizador si aún se encontraba activo, como medida de seguridad en caso de que el ataque no haya conectado con el jugador.

5.14.1.4. Behaviour Tree

El Behaviour Tree (árbol de comportamiento) es el sistema que se encarga de controlar todas las decisiones y acciones de un enemigo en el juego. Funciona evaluando constantemente la situación y eligiendo qué comportamiento ejecutar según la distancia del jugador y otras condiciones. La estructura se organiza en niveles jerárquicos donde cada nivel toma decisiones más específicas que el anterior.

El esqueleto utiliza un Behaviour Tree (*BT_AI*) con una estructura simple de tipo Selector, que evalúa dos secuencias según el estado de la *Blackboard key hasSeenPlayer?*:

²Tal como aparece en el Blueprint; se mantiene el nombre original a pesar de la inconsistencia de escritura respecto a *EndSwordTrace*.

- Si `hasSeenPlayer?` no está activa, se ejecuta la secuencia «Look Around»: el esqueleto deambula por el área cercana a su punto de aparición (`BTTask_RoamAround`) y espera 2 segundos antes de repetir el ciclo.
- Si `hasSeenPlayer?` está activa, se ejecuta la secuencia «Chase Player»: el esqueleto persigue al jugador hasta alcanzar la distancia de ataque (`BTTask_ChaseB4Attack`) y luego ejecuta su ataque con espada (`BT_TaskSWORDAttack`). El detalle de cada una de estas tareas se describe a continuación.

El Blackboard es una estructura de datos compartida utilizada por el sistema de inteligencia artificial de Unreal Engine para almacenar información que puede ser accedida y modificada tanto por el Behaviour Tree como por el AI Controller. Funciona como una pizarra donde se escriben y leen valores que representan el estado actual de la IA. Este sistema permite que diferentes componentes de la IA trabajen con la misma información sin necesidad de comunicarse directamente entre sí.

El Blackboard asociado (`BBD_AI`) contiene las siguientes *keys*:

1. `SelfActor` (Object): referencia al propio enemigo.
2. `hasSeenPlayer?` (Bool): indica si el enemigo ha detectado al jugador.
3. `IsDead?` (Bool): indica si el enemigo está muerto.
4. `spawnPoint` (Vector): posición de origen, utilizada como punto de patrulla.
5. `distanceToPlayer` (Float): distancia actual respecto al jugador.

5.14.1.5. Tareas del Behaviour Tree

- `BTTask_RoamAround` selecciona, mediante `Get Random Reachable Point In Radius`, un punto alcanzable aleatorio dentro de un radio de 500 unidades alrededor de la posición guardada en `spawnPoint`, y mueve al esqueleto hacia dicho punto con un radio de aceptación de 120 unidades.
- `BTTask_ChaseB4Attack` mueve al enemigo hacia el jugador hasta alcanzar una distancia determinada por la variable `Acceptance Radius`, finalizando la tarea al llegar o al fallar el movimiento. Esta tarea es compartida con el esqueleto mago, el cual altera su comportamiento mediante la variable `infiniteRange` (descrito en la sección correspondiente a dicho enemigo); el esqueleto normal no la utiliza.
- `BT_TaskSWORDAttack` gestiona el ataque cuerpo a cuerpo del esqueleto en dos partes. Al activarse la tarea (`ReceiveExecuteAI`), marca la variable interna `onRotate` como `true` y reproduce el montage de ataque (`MO_Attack`) sobre el esqueleto; en el instante en que la animación lo indica (*Animation Notify*), `onRotate` se marca nuevamente como `false`. Por otro lado, mientras la tarea está activa (`ReceiveTickAI`), si `onRotate` es `true`, el esqueleto rota suavemente hacia la posición del jugador en cada fotograma, interpolando su rotación actual hacia la calculada. De esta forma, el esqueleto se orienta hacia el jugador mientras se prepara para golpear, pero deja de hacerlo justo en el instante del

impacto, evitando que el golpe se desvíe por una rotación a mitad de la animación. Al completarse el montaje, la tarea finaliza exitosamente.

5.14.2. Esqueleto mago

El esqueleto mago (`BP_SkeletonMage`) comparte la base estructural y gran parte de la lógica del esqueleto normal, descrita en la sección anterior, por lo que a continuación solo se detallan las diferencias respecto a dicho enemigo.

5.14.2.1. Estructura y detección

La jerarquía de componentes es equivalente a la del esqueleto normal: cápsula de colisión con barra de vida, malla esquelética con componentes de escena para el trace de impacto cuerpo a cuerpo (`StartTrace/EndTrace`), componente de movimiento y `PawnSensing`.

La detección del jugador, sin embargo, incorpora una verificación adicional. Al activarse el evento `On See Pawn`, antes de marcar al jugador como detectado se comprueba que la diferencia de altura (eje Z) entre el mago y el jugador no supere las 300 unidades. Esta verificación evita que el mago detecte y reaccione al jugador cuando este se encuentra en un piso o nivel de altura distinto.

A diferencia del esqueleto normal, el mago no reproduce un sonido al morir.

5.14.2.2. Behaviour Tree

El Behaviour Tree del mago reutiliza la secuencia «Look Around» del esqueleto normal (`BTTask_RoamAround` + espera de 2 segundos) cuando no ha detectado al jugador. Una vez detectado, en lugar de una única secuencia de persecución y ataque, el mago evalúa un segundo Selector que distingue dos comportamientos según la distancia al jugador, mantenida actualizada mediante un servicio (`BTS_MageUpdt`) que corre en paralelo a ambas secuencias y que, en cada fotograma, calcula la distancia entre el mago y el jugador y la almacena en la *Blackboard key distanceToPlayer*:

- **Secuencia «FarSeq»** (distancia mayor a 280 unidades): el mago se acerca al jugador mediante `BTTask_ChaseB4Attack` —la misma tarea utilizada por el esqueleto normal—, pero con la variable `infiniteRange` activada. Esto provoca que el `AIMoveTo` subyacente reciba un radio de aceptación extremadamente amplio, por lo que la tarea se da por completada de inmediato sin que el mago efectivamente se desplace hacia el jugador; en la práctica, el mago no persigue al jugador en esta secuencia. A continuación, ejecuta `MagicAttack`, descrito más abajo.
- **Secuencia «CloseSeq»** (distancia menor o igual a 280 unidades): el mago ejecuta directamente `CloseMage`, su ataque cuerpo a cuerpo, descrito más abajo.

La decisión entre ambas secuencias se evalúa mediante la tarea `Task_MageDist`, la cual comprueba si la distancia actual al jugador (*Blackboard key distanceToPlayer*) se encuentra

dentro de un rango definido por las variables `min_dist` y `max_dist` propias de cada secuencia, finalizando con éxito o fracaso según corresponda.

5.14.2.3. Ataque a distancia (**MagicAttack**)

Al activarse esta tarea, se marca la variable `onRotate` como `true` y se reproduce el montage `M0_MagicAttack`. Mientras `onRotate` es `true` (evaluado en cada fotograma, en `ReceiveTickAI`), el mago interpola su rotación hacia la posición del jugador, de forma idéntica al sistema de orientación visto en el ataque del esqueleto normal. En el instante indicado por un *Animation Notify* del montage, `onRotate` se marca como `false` —deteniendo la rotación— y se instancia un proyectil (`BP_SkeletonBall`) desde una posición desplazada 300 unidades hacia adelante respecto al mago, con este último asignado como su dueño (*Owner*). El proyectil se configura con una velocidad de 800, sin influencia de gravedad, sin comportamiento de persecución (*Homing* desactivado), y un daño base de 15. Al completarse el montage, la tarea finaliza exitosamente.

5.14.2.4. Ataque cuerpo a cuerpo (**CloseMage**)

Cuando el jugador se encuentra a corta distancia, el mago recurre a un ataque cuerpo a cuerpo con su cayado. La tarea `CloseMage` marca `onRotate` como `true`, reproduce el montage `M0_CloseStaff`, y rota hacia el jugador mediante el mismo mecanismo de interpolación que el resto de las tareas del mago. Al completarse el montage, `onRotate` se restablece a `false` y la tarea finaliza con éxito, siguiendo el mismo patrón de montage-completado-finalización empleado en el resto de las tareas de ataque del juego.

Al igual que en los ataques cuerpo a cuerpo del esqueleto normal, la ventana de detección está delimitada por un *Anim Notify State* en el montage `M0_CloseStaff`, que dispara `BegomSwordTrace` y `EndSwordTrace`. Este mecanismo no aplica al ataque a distancia (`MagicAttack`), cuya detección de impacto recae en el propio proyectil al colisionar.

La detección de impacto de este ataque reutiliza el mismo sistema de *trace* por temporizador descrito para el esqueleto normal (`BegomSwordTrace`, `Damage Trace` y `EndSwordTrace`), empleando los componentes `StartTrace` y `EndTrace` del mago. A diferencia del esqueleto normal, donde el daño se obtiene de una variable (`skeleton_damage`), en el mago el daño del ataque cuerpo a cuerpo está definido como un valor fijo de 11.

5.14.3. Esqueleto caballero

El esqueleto caballero (`BP_KnightSkele`) comparte la estructura de componentes y gran parte del comportamiento base del esqueleto normal —cápsula de colisión con barra de vida, malla esquelética con componentes de escena para el *trace* de impacto (`StartTrace/EndTrace`), componente de movimiento y `PawnSensing`—, por lo que a continuación solo se detallan las diferencias respecto a dicho enemigo. Este enemigo se ubica en el nivel `Lvl_PreBoss`, donde —junto con el esqueleto normal— alimenta la métrica de distancia previa al combate contra el jefe.

5.14.3.1. Detección y registro de distancia

Al igual que el esqueleto normal, la barra de vida rota en cada fotograma (`ReceiveTick`) para orientarse hacia la cámara del jugador.

La detección del jugador mediante el evento `On See Pawn` de `PawnSensing` invoca `hasseen`, igual que en los demás esqueletos, e inicia el mismo temporizador en bucle de 0.5 segundos que dispara el evento `REGISTER_DISTANCE`, descrito en la sección del esqueleto normal. La única diferencia es que, naturalmente, la distancia que se calcula y registra corresponde a la posición del esqueleto caballero respecto al jugador, en lugar de la del esqueleto normal.

5.14.3.2. Recepción de daño y muerte

El procesamiento de daño sigue el mismo patrón general que el esqueleto normal (`hasseen` sobre el causante, resta a `health`, actualización de la barra de vida y destrucción al llegar a cero, con la consecuente llamada a `RemoveWidget` sobre el jugador). A diferencia del esqueleto normal, el caballero no reproduce ningún sonido de muerte, igual que el esqueleto mago.

5.14.3.3. Ataque cuerpo a cuerpo (trace por temporizador)

Como en los demás enemigos melee, la ventana de detección está delimitada por un *Anim Notify State* en cada montage de ataque, que dispara `BegomSwordTrace` y `EndSwordTrace`.

El esqueleto caballero también posee, heredado del mismo patrón del esqueleto normal, un ataque cuerpo a cuerpo basado en *trace* por temporizador (`BegomSwordTrace`, `DamageTrace`, `EndSwordTrace`), idéntico en su funcionamiento: temporizador en bucle de 0.01 segundos, *Sphere Trace* de radio 30 entre `StartTrace` y `EndTrace`, y detención inmediata del temporizador tras el primer impacto exitoso. La única diferencia es el nombre de la variable de daño utilizada, `KnightDamage`, en lugar de `skeleton_damage`.

5.14.3.4. Behaviour Tree

El Behaviour Tree del caballero reutiliza la secuencia «Look Around» (`BTTask_RoamAround` + espera de 2 segundos) cuando no ha detectado al jugador, y la tarea `BTTask_ChaseB4Attack` —sin la variable `infiniteRange`, al igual que el esqueleto normal— para acercarse a él una vez detectado. A diferencia de los esqueletos anteriores, tras la persecución el caballero no ejecuta una única tarea de ataque, sino que entra a un nodo compuesto personalizado llamado «Alternating Selector», con dos ramas: `Attack 1` (tarea `KnightAttack1` seguida de una espera de 1 segundo) y `Attack 2` (tarea `KnightAttack2` seguida de una espera de 1 segundo).

5.14.3.4.1. Selector alternante de ataques

El nodo «Alternating Selector» corresponde a la clase `UBTComposite_RandomSelector`, implementada en C++. A pesar de que el nombre de la clase sugiere una selección aleatoria, su lógica (`GetNextChildHandler`) no es aleatoria: en cada ejecución elige el hijo siguiente al último ejecutado mediante el operador módulo sobre la cantidad de hijos (`((LastExecutedChild + 1) % GetChildrenNum())`), partiendo del hijo 0 la primera vez. Es

decir, el nodo alterna de forma determinística entre **Attack 1** y **Attack 2** en cada ciclo, en lugar de elegir entre ellos al azar.

5.14.3.4.2. Tareas **KnightAttack1** y **KnightAttack2**

Ambas tareas comparten una misma estructura. Al activarse (**ReceiveExecuteAI**), se marca **onRotate** como **true** y se guarda una referencia casteada al propio caballero (**SkelePawn**), utilizada luego en el cálculo del *trace* de daño. A continuación se reproduce un *Animation Montage*: **MO_AvanceV3** con velocidad de reproducción 1.0 en **KnightAttack1**, y **MO_AOEKnightV3** con velocidad de reproducción 1.2 en **KnightAttack2**.

Al completarse el montaje (**OnCompleted**), la tarea finaliza con éxito (**FinishExecute**). En el instante señalado por un único *Animation Notify* del montaje (**OnNotifyBegin**), se marca **onRotate** como **false** —deteniendo la rotación hacia el jugador, igual que en el resto de las tareas de ataque del juego— y, a diferencia del ataque con espada por temporizador, se ejecuta un único *Box Trace* (sin temporizador en bucle) entre dos puntos ubicados 150 y 225 unidades respectivamente delante del caballero, en la dirección de su vector hacia adelante. La forma de la caja del trace difiere entre ambas tareas: en **KnightAttack1** es un cubo de $50 \times 50 \times 50$ unidades, mientras que en **KnightAttack2** es una caja de $30 \times 80 \times 50$ unidades, más ancha. De detectarse una colisión válida, se aplica daño mediante **ApplyDamage**, con un valor fijo de 20 en ambas tareas (a diferencia del **KnightDamage** variable utilizado en el ataque con espada).

Mientras la tarea está activa (**ReceiveTickAI**), y siguiendo el mismo patrón visto en el resto de los enemigos, el caballero rota suavemente hacia el jugador mediante **RInterpTo** (con una velocidad de interpolación de 3.0) mientras **onRotate** sea **true**, deteniéndose justo en el instante del impacto.

5.15. Jefe (**BP_Slime**)

5.15.1. Modelo y animaciones

El modelo y las animaciones del jefe fueron creados desde cero en Blender, en lugar de recurrir a assets externos como en el resto de los enemigos. La motivación principal fue contar con un control total sobre el resultado, de modo que el modelo y sus animaciones pudieran ajustarse específicamente a los requerimientos de *gameplay* del jefe —en particular, a los *Anim Notify States* que delimitan las ventanas de ataque y a las distintas formas que necesita adoptar durante el combate.

Dado que el slime no posee un esqueleto tradicional, gran parte de sus deformaciones se modelaron mediante *shape keys*, tanto para las distintas formas que puede adoptar el cuerpo (por ejemplo, su forma de charco, o su forma donde de espigas) como para las acciones propias de cada ataque.

5.15.2. Componentes y variables

La jerarquía de componentes de **BP_Slime** es la siguiente:

1. `CollisionCylinder`: cápsula de colisión, raíz del actor.
2. `PULL_OUT`: componente de colisión de tipo esfera.
3. `CharacterMesh0`: malla esquelética del jefe.
 1. `StartTracePos`: componente de escena, punto de inicio del *trace* de impacto cuerpo a cuerpo.
 2. `EndTracePos`: componente de escena, punto de fin del *trace* de impacto cuerpo a cuerpo.
 3. `Arrow`: componente de flecha, referencia direccional.
4. `PlayerMetrics` (`PlayerMetricsComponent`): descrito en Sección 5.16.
5. `CharMoveComp`: componente de movimiento de personaje.

Adicionalmente, `BP_Slime` implementa la interfaz `BPI_Lockable`, consultada por el sistema de fijado de objetivo del jugador (*Target Lock*, descrito en la sección correspondiente) para obtener el punto exacto al que debe apuntar la cámara mientras el jefe está fijado. Esto permite compensar el desplazamiento visual que sufre el jefe durante su animación de ataque de charco, donde se hunde visualmente sin que ese desplazamiento se traduzca en un movimiento real del actor. Su función `Get Lock On Target` retorna una posición desplazada hacia abajo respecto a la ubicación del actor: 300 unidades en el eje Z si `bIsCharco` es `true`, o 100 unidades si es `false`.

Las variables propias del Blueprint son:

1. `health` (Float): vida actual del jefe.
2. `max_health` (Float): vida máxima del jefe.
3. `progressbarref` (Progress Bar): referencia al widget de la barra de vida.
4. `bIsCharco` (Bool): indica si el jefe se encuentra en su forma de charco.

5.15.3. AI Controller

El AI Controller del jefe (`BP_SlimeBossController`) expone una función llamada `ActivateBossController`, invocada desde una zona de activación (*trigger*) al inicio de la arena del jefe. Al ejecutarse, esta función arranca el Behavior Tree (`BT_BaseSlimeBoss`), registra la referencia al jugador en el Blackboard (`PlayerActor`) e invoca `ApplyPreCombatAdaptation` sobre el `PlayerMetricsComponent`, aplicando los ajustes de pesos de ataque derivados de las métricas recolectadas durante la fase previa al combate. Finalmente, vuelca el estado resultante de los pesos a archivo mediante `DumpFinalTreeToFile`, tal como se describió en la sección Sección 5.16.

5.15.4. Blackboard

El Blackboard del jefe (`BBD_SlimeBoss`) contiene las siguientes *keys*:

1. `SelfActor`: Tipo *Actor*. Referencia al propio actor controlado por la IA, utilizada para acceder a sus datos y ejecutar acciones sobre sí mismo.

2. **PlayerActor**: Tipo *Actor*. Referencia al jugador detectado por la IA, empleada para seguimiento, persecución y ataques.
3. **PlayerDistance**: Tipo *Float*. Almacena la distancia actual entre la IA y el jugador.
4. **LastAttack**: Tipo *Integer*. Guarda el identificador o índice del último ataque ejecutado por el enemigo.
5. **lastAttackCount**: Tipo *Integer*. Contador utilizado para registrar la cantidad de ataques realizados o secuencias de ataque recientes.
6. **AttackWeightsJSON**: Tipo *String*. Contiene datos de configuración de pesos de ataques almacenados en formato JSON.
7. **CloseRange**: Tipo *Float*. Distancia considerada como rango cercano para la toma de decisiones de combate.
8. **FarRange**: Tipo *Float*. Distancia considerada como rango lejano para la toma de decisiones de combate.
9. **0-KEY**: Tipo *Float/Integer*. Valor asociado a la clave «0» dentro de una estructura de configuración o tabla de pesos.
10. **FAR-KEY**: Tipo *Float/Integer*. Valor asociado a la clave de comportamiento o ataque de larga distancia.
11. **BossChaseDur**: Tipo *Float*. Duración durante la cual el jefe mantiene el estado de persecución antes de cambiar de comportamiento.

5.15.5. Árbol de comportamiento

5.15.5.1. BTComposite_RandomSelector (C++)

5.15.5.2. Tareas de ataque

El jefe cuenta con nueve tareas de ataque distintas, identificadas mediante el enum `EBossAttackType` (`BA_BasicAttack`, `BA_AOEAttack`, `BA_HeavyAttack`, `BA_ProjectilAttack`, `BA_BossChase`, `BA_Poddle`, `BA_HomingAttack`, `BA_WhipAttack`, `BA_WallAttack`), seleccionadas por el `BTComposite_RandomSelector` descrito en la sección anterior.

5.15.5.2.1. Ataque básico (`BA_BasicAttack`)

5.15.5.2.2. Ataque de área (`BA_AOEAttack`)

5.15.5.2.3. Ataque pesado (`BA_HeavyAttack`)

El ataque pesado corresponde a un ataque con salto.

5.15.5.2.4. Ataque de proyectil (`BA_ProjectilAttack`)

5.15.5.2.5. Persecución (`BA_BossChase`)

5.15.5.2.6. Charco (`BA_Poddle`)

Esta tarea hace uso de la forma de charco del jefe, modelada mediante *shape keys* (ver Sección 5.15.1).

5.15.5.2.7. Ataque homing (**BA_HomingAttack**)

5.15.5.2.8. Ataque de látigo (**BA_WhipAttack**)

El ataque de látigo corresponde a un ataque tipo espada/látigo.

5.15.5.2.9. Ataque de muro (**BA_WallAttack**)

5.15.6. **PlayerMetricsComponent**

5.15.7. Secuencia de introducción

5.16. Sistema de métricas y telemetría

De forma transversal a varios de los sistemas descritos en este capítulo, el juego incorpora una infraestructura de recolección y uso de métricas sobre el comportamiento del jugador, implementada principalmente en dos clases de C++: **SlimeGameInstance**, encargada de recolectar las métricas y persistirlas, y **PlayerMetricsComponent**, anclado al jefe, encargado de registrar la efectividad de sus ataques y de traducir las métricas recolectadas en ajustes sobre su comportamiento.

5.16.1. Variables de **SlimeGameInstance**

Como se mencionó, **SlimeGameInstance** recolecta sus datos en dos fases paralelas, controladas mediante la variable **isInCombat**: una fase previa al combate (en **Lvl_PreBoss**), condicionada además a que **bMetricsEnabled** esté activo, y una fase durante el combate contra el jefe, cuyas variables llevan el prefijo **Combat**. A continuación se detallan las variables principales, agrupadas por categoría.

Cabe precisar de antemano el rol efectivo de estas variables con prefijo **Combat** dentro del sistema de adaptación. Si bien se registran de forma paralela a sus contrapartes sin prefijo —mediante el mismo flag **isInCombat**— y forman parte del diseño original de la infraestructura de métricas, que contemplaba adaptar también el comportamiento del jefe durante el combate en base a estas métricas, dicha adaptación en tiempo real no se llegó a implementar. La adaptación pre-combate (**ApplyPreCombatAdaptation** y sus subfunciones, ver Sección 5.16.6) lee exclusivamente las variables sin prefijo; ningún punto del sistema consulta las variables **Combat*** para tomar decisiones. Las únicas excepciones, es decir, las únicas métricas recolectadas durante el combate que sí tienen un efecto real sobre el comportamiento del jefe, son el registro de aciertos por tipo de ataque (**TotalAttemptsPerType** y **SuccessfulHits**, en **PlayerMetricsComponent**), que alimenta **ApplyInCombatSuccessfulHits**, y el monitoreo de curación en combate vía **ApplyInCombatHealing**, que compara la vida actual del jugador contra **AverageHealingHP** —calculada en la fase pre-combate— para ajustar la presión de ataque cuando el jugador entra en su rango habitual de curación.

Distancia

1. **DistanceAccum**, **DistanceSamples**: acumulador y contador de muestras de distancia (pre-combate).

2. **AverageDistance**: promedio de distancia, recalculado en cada muestra (pre-combate).
3. **CombatDistanceAccum**, **CombatDistanceSamples**, **CombatAverageDistance**: equivalentes durante el combate.

Ataques

1. **MeleeAttacks**, **RangedAttacks**: conteo de ataques melee y a distancia (pre-combate).
2. **CombatMeleeAttacks**, **CombatRangedAttacks**: equivalentes durante el combate.
3. **MeleeAttacksInRangedZone**, **RangedAttacksInRangedZone**: conteo de cada tipo de ataque realizado dentro de una zona de rango.

Curación

1. **HealsAtHighHP**, **HealsAtMidHP**, **HealsAtLowHP**: conteo de curaciones según el rango de vida del jugador al curarse (pre-combate).
2. **HealingHPAccum**, **HealingCount**: acumulador y contador para el promedio de vida al curarse.
3. **AverageHealingHP**: promedio del porcentaje de vida del jugador al momento de curarse.
4. **CombatHealsAtHighHP**, **CombatHealsAtMidHP**, **CombatHealsAtLowHP**, **CombatHealingHPAccum**, **CombatHealingCount**, **CombatAverageHealingHP**: equivalentes durante el combate.

Esquivas y daño

1. **TotalDodges**, **TotalDodgesTankZone**: conteo total de esquivas, y de aquellas realizadas en la zona de tanque (pre-combate).
2. **SuccessfulDodges**: conteo de esquivas marcadas explícitamente como exitosas.
3. **DamageTaken**, **TotalDamageReceived**: conteo de impactos recibidos y daño total acumulado (pre-combate).
4. **CombatTotalDodges**, **CombatSuccessfulDodges**: equivalentes durante el combate.
5. **CombatDamageTaken**, **CombatTotalDamageReceived**: equivalentes durante el combate.
6. **LateralDodgesFromDash**, **BackwardDodgesFromDash**: esquivas laterales o hacia atrás frente a el ataque de embestida del caballero esqueleto.
7. **DodgesFromDelayAttack**, **DodgesFromDelayAttack_TankZone**: esquivas frente a un ataque con retardo, en general y en zona de tanque.
8. **CombatDodgesFromDelayAttack**: equivalente durante el combate.

Zona de tanque

1. **isInTankZone**: flag de si el jugador se encuentra actualmente en la zona de tanque.
2. **DamageTakenInTankZone**: conteo de daño recibido en dicha zona.

Estudio

1. **bAdaptiveEnabled**: condición experimental del jugador (**true** = adaptativo, **false** = control).
2. **bMetricsEnabled**: habilita o no la recolección durante el nivel.
3. **SessionId**: identificador de sesión, generado a partir de fecha y hora.

4. `AttemptNumber`: número de intento actual contra el jefe.

5.16.2. Funciones de registro (`SlimeGameInstance`)

Sobre las variables anteriores operan las siguientes funciones, todas siguiendo el mismo patrón: bifurcan su comportamiento según `isInCombat`, actualizando el conjunto de variables de la fase correspondiente.

1. `RegisterDistance(Distance)`: acumula `Distance` en `DistanceAccum` (o su equivalente de combate), incrementa el contador de muestras, y recalcula el promedio dividiendo el acumulador entre el número de muestras.
2. `RegisterMeleeAttack()` y `RegisterRangeAttack()`: incrementan `MeleeAttacks` o `RangedAttacks` respectivamente (o sus equivalentes de combate); adicionalmente, si `isInRangedZone` es verdadero, incrementan también `MeleeAttacksInRangedZone` o `RangedAttacksInRangedZone` según corresponda.
3. `RegisterHealingMoment(PlayerHealth, PlayerMaxHealth)`: calcula $(\text{PlayerHealth} / \text{PlayerMaxHealth}) * 100$, incrementa `HealingCount` y acumula el porcentaje en `HealingHPAccum` para recalcular `AverageHealingHP`; según el porcentaje resultante, incrementa `HealsAtHighHP` ($> 70\%$), `HealsAtMidHP` ($> 30\%$) o `HealsAtLowHP` ($\leq 30\%$).
4. `RegisterTotalDodge()`: incrementa `TotalDodges`; si `isInTankZone` es verdadero, incrementa además `TotalDodgesTankZone`.
5. `RegisterDodgeResult(bWasSuccessful)`: si `bWasSuccessful` es verdadero, incrementa `SuccessfulDodges`. Si es falso, la función no realiza ninguna acción.
6. `RegisterDamageTaken(Damage)`: incrementa `DamageTaken` y acumula `Damage` en `TotalDamageReceived`.
7. `RegisterDashDodge(bWasLateral)`: incrementa `LateralDodgesFromDash` o `BackwardDodgesFromDash` según `bWasLateral`. A diferencia de las anteriores, no bifurca por `isInCombat`; solo depende de `bMetricsEnabled`.
8. `RegisterDodgeFromDelayAttack()`: incrementa `DodgesFromDelayAttack` (o `CombatDodgesFromDelayAttack` en combate); si `isInTankZone` es verdadero, incrementa además `DodgesFromDelayAttack_TankZone` en ambos casos.
9. `RegisterDamageTakenInTankZone()`: incrementa `DamageTakenInTankZone`. No bifurca por `isInCombat`, solo depende de `bMetricsEnabled`.

5.16.3. Registro de ataques del jefe (`PlayerMetricsComponent`)

`PlayerMetricsComponent` lleva un registro independiente mediante dos mapas indexados por tipo de ataque (`EBossAttackType`): `TotalAttemptsPerType` y `SuccessfulHits`.

`RegisterBossAttackAttempt(AttackType)` incrementa el contador de intentos correspondiente en `TotalAttemptsPerType`, además de un contador global `TotalAttacksPerformed`. Cada 15 ataques realizados (`TotalAttacksPerformed % 15 == 0`), se dispara automáticamente `ApplyInCombatSuccessfulHits`.

`RegisterBossAttackHit(AttackType)` incrementa el contador de aciertos correspondiente en `SuccessfulHits`.

5.16.4. Adaptación pre-combate

Al finalizar la fase previa al combate, `ApplyPreCombatAdaptation` analiza las variables de `SlimeGameInstance` y ajusta los pesos de selección de ataques del jefe (mecanismo descrito en Sección 5.16.6) en cuatro dimensiones independientes, llamando para ello a las funciones `ApplyPreCombatDistance`, `ApplyPreCombatMeleeVsRanged`, `ApplyPreCombatDodges` y `ApplyPreCombatTankZone`, descritas a continuación. Todo el sistema de adaptación solo se ejecuta si `IsAdaptiveEnabled` retorna verdadero, lo cual depende de `bAdaptiveEnabled`.

Distancia (`ApplyPreCombatDistance`): requiere `DistanceSamples` $\geq$ 20. Si `AverageDistance` $<$ 550, reduce en 50 la variable de Blackboard `CloseRange`. Si `AverageDistance` $>$ 1100, reduce en 75 `FarRange` e incrementa en 2 segundos `BossChaseDur`. Entre ambos umbrales, no hay ajuste.

Melee versus a distancia (`ApplyPreCombatMeleeVsRanged`): requiere `MeleeAttacks` + `RangedAttacks` $\geq$ 5. Calcula `RangedRatio` = `RangedAttacks` / `Total`. Si `RangedRatio` $>$ 0.65, incrementa en 15 los pesos de `BA_Poddle`, `BA_BossChase` y `BA_HeavyAttack`. Si `RangedRatio` $<$ 0.35, incrementa en 15 los pesos de `BA_AOEAttack` y `BA_WhipAttack`. Adicionalmente, si `MeleeAttacksInRangedZone` + `RangedAttacksInRangedZone` $\geq$ 3, incrementa en 10 el peso de `BA_AOEAttack` si predominó el melee en zona de rango, o el de `BA_Poddle` si predominó el ataque a distancia.

Esquiva (`ApplyPreCombatDodges`): requiere `TotalDodges` $\geq$ 5. Calcula `DodgeRatio` = `DodgesFromDelayAttack` / `TotalDodges`. Si `DodgeRatio` $>$ 0.6, incrementa en 15 el peso de `BA_BasicAttack`. Si `DodgeRatio` $<$ 0.3, incrementa en 15 los pesos de `BA_AOEAttack`, `BA_WallAttack` y `BA_HeavyAttack`. Entre ambos umbrales, no hay ajuste.

Zona de tanque (`ApplyPreCombatTankZone`): evalúa dos aspectos. Si `LateralDodgesFromDash` + `BackwardDodgesFromDash` $\geq$ 3, incrementa en 10 el peso de `BA_WhipAttack` (si predominaron las laterales) o de `BA_HeavyAttack` (si predominaron las de retroceso). Si `TotalDodgesTankZone` $\geq$ 3, compara si `DodgesFromDelayAttack_TankZone` * 2 $\geq$ `TotalDodgesTankZone`: de ser así, incrementa en 10 el peso de `BA_BasicAttack`; en caso contrario, incrementa en 10 los pesos de `BA_HeavyAttack` y `BA_WhipAttack`.

5.16.5. Adaptación durante el combate

`ApplyInCombatSuccessfulHits`, disparada cada 15 ataques del jefe, evalúa cada tipo de ataque con `TotalAttemptsPerType` $\geq$ 3, calculando `Ratio` = `SuccessfulHits` / `TotalAttemptsPerType`. Si `Ratio` $>$ 0.6, incrementa el peso de ese ataque en 15. Si `Ratio` $<$ 0.3, lo reduce en 15. Entre ambos umbrales, no hay ajuste.

`ApplyInCombatHealing(PlayerHealthPercent)` requiere `HealingCount >= 2`. Compara `PlayerHealthPercent` contra `AverageHealingHP` con un margen de 10 puntos porcentuales. Al entrar en dicho margen (controlado por la variable interna `bIsInHealingRange`, para evitar reaplicar el ajuste en cada frame), incrementa en 15 los pesos de `BA_BasicAttack`, `BA_HeavyAttack`, `BA_HomingAttack` y `BA_Poddle`. Al salir del margen, revierte los mismos ajustes en -15 .

5.16.6. Mecanismo de pesos de ataque

Los pesos de selección de ataques del jefe, utilizados por el nodo compuesto de selección aleatoria ponderada del Behavior Tree, se almacenan como una única cadena en formato JSON dentro de la variable de Blackboard `AttackWeightsJSON`, en lugar de como variables individuales. `SetAttackWeight(AttackType, NewWeight)` deserializa dicha cadena, actualiza el peso correspondiente al tipo de ataque indicado, y vuelve a serializarla y almacenarla. `GetAttackWeight(AttackType)` realiza el proceso inverso, retornando el peso actual de un tipo de ataque (o -1 si no se encuentra).

5.16.7. Persistencia de datos para el estudio

Los datos se persisten en archivos JSON dentro de `Saved/StudyLogs`:

1. `DumpMetricsToFile (SlimeGameInstance):` vuelca las variables de la fase pre-combate (`AverageDistance`, `DistanceSamples`, `MeleeAttacks`, `RangedAttacks`, `MeleeAttacksInRangedZone`, `RangedAttacksInRangedZone`, `TotalDodges`, `SuccessfulDodges`, `AverageHealingHP`, `HealingCount`, `LateralDodgesFromDash`, `BackwardDodgesFromDash`, `DodgesFromDelayAttack`, `DamageTakenInTankZone`, `TotalDodgesTankZone`, `DodgesFromDelayAttack_TankZone`) a `session_{SessionId}_{condición}_metrics.json`.
2. `DumpCombatResultToFile(bPlayerWon) (PlayerMetricsComponent):` vuelca `bPlayerWon`, `AttemptNumber` y el contenido de `AttackWeightsJSON` a `session_{SessionId}_{condición}_win.json` o `_retry_{AttemptNumber}.json`, según corresponda.
3. `DumpFinalTreeToFile (PlayerMetricsComponent):` vuelca únicamente `AttackWeightsJSON` a `session_{SessionId}_{condición}_tree.json`.

En todos los casos, `SessionId` (generado a partir de fecha y hora) y la condición experimental (`adaptive` o `control`, según `bAdaptiveEnabled`) identifican tanto el contenido como el nombre del archivo.

5.17. Niveles

5.17.1. Lvl_Tutorial

5.17.2. Lvl_PreBoss

5.17.3. Lvl_ThirdPerson

5.17.4. Elementos de nivel

5.17.5. Widgets de tutorial

5.18. Widgets generales (MainMenu, WinnerScreen...)

Capítulo 6

Prueba de concepto y evaluación

Capítulo 7

Conclusión y trabajo futuro

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim admodum doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Bibliografía

Anexo A

Anexo

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim admodum doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim admodum doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.